

SYSTEM S2 — SOFTWARE DESIGN DOCUMENT

Employee master data, departments & positions, disciplinary/assets/ guarantors/offboarding, payroll runs & payslips, leave & attendance — the system of record for everyone who works at the hotel

CONSOLIDATES FORMER MODULES

M2 Human Resource Service

M3 Payroll Service

M4 Leave & Attendance Service

DOCUMENT INFORMATION

Document type	Software Design Document — Developer Edition
Intended audience	The developer (or team) building System 2 — Workforce & Payroll. Read S0 first.
Version	1.0
Date	2026-06-15
Companion documents	S0 Shared Platform Reference · S1 Identity & Access Platform · S3 Hospitality Operations · S4 Finance & BI

Table of Contents

1. Purpose, Scope & Responsibilities	4
1.1 Why S2 exists	4
1.2 Out of scope	4
1.3 Relationship to the original M2 / M3 / M4 services	5
2. Internal Architecture	7
2.1 Functional areas	7
2.2 The ID6 simplification	8
2.3 Folder structure	9
3. Data Model	11
3.1 employees	11
3.2 departments	12
3.3 positions	13
3.4 disciplinary_records	13
3.5 employee_assets	13
3.6 asset_types	14
3.7 guarantors	14
3.8 offboarding_records	14
3.9 Payroll & Leave tables	15
3.10 Platform plumbing tables	21
4. API Endpoint Catalog	22
4.1 HR module	22
4.2 Payroll module	23
4.3 Leave & Attendance module	24
5. Core Business Logic	25
5.1 Payroll run lifecycle overview	25
5.2 Earnings components	27
5.3 Deductions and the ERCA tax schedule	27
5.4 Overtime pay	28

5.5 Pension contributions	28
5.6 Severance calculation	29
5.7 Annual leave accrual	29
5.8 Attendance and overtime approval workflow	30
6. Role-Based Access Control	31
6.1 Permission catalog	31
6.2 System roles — S2 access summary	32
6.3 CheckPermission middleware contract	33
6.4 Permission seeding	33
7. Employee & Payroll Run Lifecycle	34
7.1 Section A — Employee lifecycle (<code>employees.employment_status</code>)	35
7.2 Section B — Payroll run lifecycle (<code>payroll_runs.status</code>)	35
8. Event-Driven Integration	36
8.1 Events emitted by S2	36
8.2 Events consumed by S2	36
8.3 Synchronous calls	37
8.4 Outbox mechanics	37
8.5 Idempotency mechanics (D5)	38
9. Disciplinary Actions, Assets, Guarantors & the Dead File Offboarding Workflow	39
9.1 Disciplinary escalation	39
9.2 Asset assignment and clearance	39
9.3 Guarantor registration	40
9.4 The Dead File offboarding workflow	40
9.5 Dead File retention rule	40
10. Security, Secrets & Operational Conventions	41
10.1 Platform conventions (D1–D14) as applied to S2	41
10.2 Environment configuration (<code>.env</code>)	41
11. Non-Functional Requirements & Testing	43
12. Integration Contract Summary	44
12.1 What S2 provides	44
12.2 What S2 depends on	44
12.3 Build-order implication	44

NOTE

This document is the complete build specification for **System 2 — Workforce & Payroll**, one Laravel 11 application backed by one MySQL database (`wh_s2_db`). S2 consolidates three former microservices — **M2 Human Resource**, **M3 Payroll**, and **M4 Leave & Attendance** — into a single modular monolith with three internal modules: **HR**, **Payroll**, and **Leave & Attendance**.

The biggest simplification from this merger is **ID6**: because employee, payroll, and leave data now live in the same database, the payroll engine reads attendance, leave balances, and overtime records with plain Eloquent queries inside the same request — there is no cross-service HTTP call to go stale, retry, or time out.

Before reading further, make sure you have read [S0 — Cross-System Integration & Platform Conventions](#), which defines the gateway, the JWT flow, the cross-system event bus, the permission-naming convention (ID9), and the platform conventions (D1–D14) referenced throughout this document by ID.

1. Purpose, Scope & Responsibilities

1.1 Why S2 exists

S2 is the **single source of truth for the workforce** — every person who works at the hotel, what they are paid, when they are off duty, and what they are owed when they leave. Concretely, S2 owns:

- **Employee master data** — personal details, employment status, job assignment (department, position), pension category, banking details, and photo.
- **Organisational structure** — departments and positions, including the reporting hierarchy used by leave-approval routing.
- **Workforce records** — disciplinary actions, company asset assignments (with an asset-type catalog), guarantor registrations, and the offboarding (“Dead File”) process.
- **Payroll** — monthly payroll runs, per-employee payroll entries, earnings and deduction line items, the ERCA progressive income-tax calculation, pension contributions, overtime pay, staff loans, and severance calculations.
- **Leave & attendance** — leave types, employee leave balances, leave requests and approvals, annual leave accrual, overtime records, and daily attendance.

All of the above lives in 22 tables inside `wh_s2_db`. S2 is the system every department head, HR officer, and payroll officer interacts with daily, and it is the system that feeds payroll-driven journal entries to S4 and employee lifecycle events to S1.

1.2 Out of scope

CONVENTION

S2 does not do any of the following — these responsibilities belong to other systems, and S2 only interacts with them through the contracts defined in S0.

Responsibility	Owned by	How S2 interacts with it
Authentication, JWT issuance/verification, RBAC enforcement	S1	Every inbound request carries a JWT verified by the gateway (S0 §3–4); S2 only checks <code>permissions[]</code> claims via <code>CheckPermission</code> middleware.
	S1	S2 emits <code>wh.events.s2.employee.creat</code>

Responsibility	Owned by	How S2 interacts with it
User accounts, login credentials, password resets		<code>ed</code> / <code>.updated</code> / <code>.archived</code> so S1 can create, sync, or deactivate the matching <code>users</code> row (S0 §5.2, ID1, ID2).
Chart of accounts, ledger, financial statements	S4	S2 posts payroll and severance journal entries via <code>POST /api/v1/journal-entries</code> (S0 §5.3) but stores no account balances itself.
Staff food & beverage consumption charges	S3	S3 calculates the staff meal charge and pushes it to S2 as a payroll deduction via <code>POST /employees/{id}/deductions</code> (ID12, S0 §5.3).
BI dashboards, KPI aggregation, scheduled reports	S4	S4 reads S2's employee, payroll-run, leave-request, and attendance data via cached <code>GET</code> endpoints (S0 §5.3–5.4, 300s TTL).
Any user interface	—	Out of scope for every system in this architecture; S2 exposes a JSON API only.

1.3 Relationship to the original M2 / M3 / M4 services

If you previously read the M2, M3, or M4 microservice specs, the following table summarises what changed when they were merged into S2.

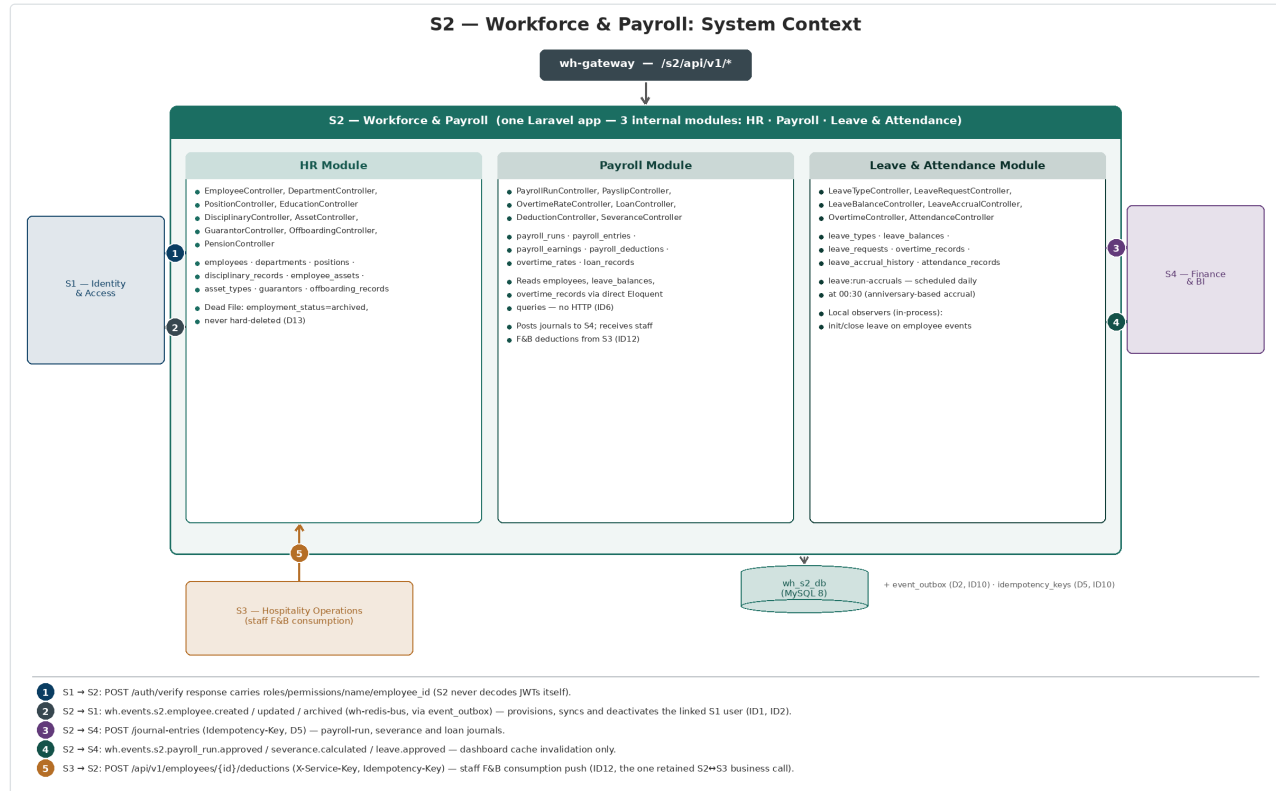
Area	Former (M2/M3/M4)	Now (S2)
Cross-module data access	M3 called M2's <code>/employees/{id}</code> and M4's <code>/employees/{id}/unused-leave-balance</code> , <code>/employees/{id}/overtime-approved</code> over HTTP	ID6 — direct Eloquent queries inside the same request; no HTTP, no staleness, no retry logic
Employee lifecycle → S1	M2 called M1's user-management API directly	S2 emits <code>employee.created</code> / <code>.updated</code> / <code>.archived</code> events; S1 consumes them (ID1, ID2)
Display name sync	Not modelled	<code>users.display_name</code> (used in JWT <code>name</code> claim) is kept in sync via the <code>employee.created</code> / <code>.updated</code> events (ID2)
Cross-system messaging	Each service had its own ad-hoc webhook conventions	Unified <code>event_outbox</code> table + <code>PublishOutboxEvents</code> job (D2, ID10)
Idempotent writes	Not modelled	<code>idempotency_keys</code> table (D5, ID10) protects <code>POST /payroll-runs/{id}/approve</code> , <code>POST /employees/{id}/loans</code> , and the inbound <code>POST /employees/{id}/deductions</code> from S3

Area	Former (M2/M3/M4)	Now (S2)
Permission names	<code>M2.*</code> , <code>M3.*</code> , <code>M4.*</code> (e.g. <code>M3.payroll.approve</code>)	Renamed to <code>S2.hr.*</code> , <code>S2.payroll.*</code> , <code>S2.leave.*</code> per ID9's <code>S{n}.{domain}.{resource}.{action}</code> convention
Severance calculation & Dead File update	M3 calculated severance, then called M2 to update the offboarding record over HTTP	In-process: <code>SeveranceService</code> reads <code>employees / leave_balances</code> directly and writes <code>offboarding_records.severance_amount</code> in the same transaction (S0 §5.5) — this column did not exist in the original M2 schema
Staff F&B deduction from S3	Not modelled in original specs	New inbound endpoint <code>POST / employees/{id}/deductions</code> , Idempotency-Key protected, Service-Key authenticated (ID12)

⋮

2. Internal Architecture

S2 is organised into three Laravel modules under a single application and database. The diagram below shows the module boundaries, their primary controllers and tables, and how S2 talks to S1, S3, and S4.



Internal architecture of System 2, showing three modules — HR, Payroll, and Leave & Attendance — inside a single Laravel application backed by the `wh_s2_db` database. The HR module contains `EmployeeController`, `DepartmentController`, `PositionController`, `DisciplinaryController`, `AssetController`, `GuarantorController`, and `OffboardingController`, operating on the `employees`, `departments`, `positions`, `disciplinary_records`, `employee_assets`, `asset_types`, `guarantors`, and `offboarding_records` tables. The Payroll module contains `PayrollRunController`, `PayslipController`, `OvertimeRateController`, `LoanController`, and `SeveranceController`, operating on `payroll_runs`, `payroll_entries`, `payroll_earnings`, `payroll_deductions`, `overtime_rates`, and `loan_records`. The Leave & Attendance module contains `LeaveTypeController`, `LeaveRequestController`, `LeaveBalanceController`, `LeaveAccrualController`, `OvertimeController`, and `AttendanceController`, operating on `leave_types`, `leave_balances`, `leave_requests`, `overtime_records`, `leave_accrual_history`, and `attendance_records`. Five numbered interaction arrows connect S2 to the rest of the platform: (1) S1 to S2, inbound JWT-verified requests through the gateway; (2) S2 to S1, employee lifecycle events (created, updated, archived) published to the event bus; (3) S2 to S4, payroll-run-approved and severance-calculated events plus a synchronous POST to `/journal-entries`; (4) S3 to S2, a synchronous POST to `/employees/{id}/deductions` for staff meal charges; (5) S4 to S2, cached read-only GET requests for employees, payroll runs, leave requests, and attendance records.

2.1 Functional areas

HR module

Concern	Controllers	Tables
Employee master data	<code>EmployeeController</code>	<code>employees</code>
Organisational structure	<code>DepartmentController</code> , <code>PositionController</code>	<code>departments</code> , <code>positions</code>

Concern	Controllers	Tables
Disciplinary actions	<code>DisciplinaryController</code>	<code>disciplinary_records</code>
Company assets	<code>AssetController</code>	<code>employee_assets</code> , <code>asset_types</code>
Guarantors	<code>GuarantorController</code>	<code>guarantors</code>
Offboarding / Dead File	<code>OffboardingController</code>	<code>offboarding_records</code>
Pension category lookup	<code>PensionController</code> (read-only helper for <code>employees.pension_category</code>)	<code>employees</code>

Payroll module

Concern	Controllers	Tables
Payroll runs & entries	<code>PayrollRunController</code>	<code>payroll_runs</code> , <code>payroll_entries</code>
Earnings & deductions	(handled within <code>PayrollRunController</code> during run generation; exposed read-only)	<code>payroll_earnings</code> , <code>payroll_deductions</code>
Payslips	<code>PayslipController</code>	<code>payroll_entries</code> , <code>payroll_earnings</code> , <code>payroll_deductions</code>
Overtime pay rates	<code>OvertimeRateController</code>	<code>overtime_rates</code>
Staff loans	<code>LoanController</code>	<code>loan_records</code>
Severance	<code>SeveranceController</code> (triggered from <code>OffboardingController</code>)	<code>offboarding_records</code> , <code>payroll_entries</code>

Leave & Attendance module

Concern	Controllers	Tables
Leave type catalog	<code>LeaveTypeController</code>	<code>leave_types</code>
Leave balances	<code>LeaveBalanceController</code>	<code>leave_balances</code>
Leave requests & approvals	<code>LeaveRequestController</code>	<code>leave_requests</code>
Annual leave accrual	<code>LeaveAccrualController</code> + <code>RunLeaveAccruals</code> command	<code>leave_accrual_history</code> , <code>leave_balances</code>
Overtime records	<code>OvertimeController</code>	<code>overtime_records</code>
Daily attendance	<code>AttendanceController</code>	<code>attendance_records</code>

2.2 The ID6 simplification

In the original three-microservice design, generating a payroll run required M3 to make outbound HTTP calls to M2 (for each employee's basic salary, pension category, and start date) and to M4 (for each employee's unused leave balance and any approved-but-unpaid overtime records). Each of those calls needed its own retry policy, timeout, and fallback behaviour, and the data could be stale by the time the payroll run actually committed.

In S2, all of that data lives in the same database. The `PayrollRunController`'s run-generation logic simply issues Eloquent queries against `employees`, `leave_balances`, and `overtime_records` — in the same

database transaction as the payroll entries it creates. There is no network call, no serialization, no stale snapshot, and no possibility of M3 succeeding while M2/M4 are down. Section 5 walks through exactly which tables are read at which step of the payroll run.

The only data that genuinely originates outside S2 during a payroll run is the staff meal deduction pushed in by S3 (ID12, Section 4.2) — and even that arrives well before the run is generated, so by the time `PayrollRunController` runs, it is just another row in `payroll_deductions`.

2.3 Folder structure

```

app/
├── Http/
│   ├── Controllers/
│   │   ├── Hr/
│   │   │   ├── EmployeeController.php
│   │   │   ├── DepartmentController.php
│   │   │   ├── PositionController.php
│   │   │   ├── DisciplinaryController.php
│   │   │   ├── AssetController.php
│   │   │   ├── GuarantorController.php
│   │   │   ├── OffboardingController.php
│   │   │   └── PensionController.php
│   │   ├── Payroll/
│   │   │   ├── PayrollRunController.php
│   │   │   ├── PayslipController.php
│   │   │   ├── OvertimeRateController.php
│   │   │   ├── LoanController.php
│   │   │   └── SeveranceController.php
│   │   └── Leave/
│   │       ├── LeaveTypeController.php
│   │       ├── LeaveBalanceController.php
│   │       ├── LeaveRequestController.php
│   │       ├── LeaveAccrualController.php
│   │       ├── OvertimeController.php
│   │       └── AttendanceController.php
│   ├── Middleware/
│   │   ├── VerifyJwt.php
│   │   ├── CheckPermission.php
│   │   ├── ServiceKeyAuthenticate.php
│   │   └── EnsureIdempotent.php
│   └── Models/
│       ├── Hr/
│       │   ├── Employee.php
│       │   ├── Department.php
│       │   ├── Position.php
│       │   ├── DisciplinaryRecord.php
│       │   ├── EmployeeAsset.php
│       │   ├── AssetType.php
│       │   ├── Guarantor.php
│       │   └── OffboardingRecord.php
│       ├── Payroll/
│       │   ├── PayrollRun.php
│       │   ├── PayrollEntry.php
│       │   ├── PayrollEarning.php
│       │   ├── PayrollDeduction.php
│       │   ├── OvertimeRate.php
│       │   └── LoanRecord.php
│       └── Leave/
│           ├── LeaveType.php
│           ├── LeaveBalance.php
│           └── LeaveRequest.php

```

```

├── OvertimeRecord.php
├── LeaveAccrualHistory.php
├── AttendanceRecord.php
├── EventOutbox.php
├── IdempotencyKey.php
├── Services/
│   ├── Hr/
│   │   └── OffboardingService.php
│   ├── Payroll/
│   │   ├── TaxCalculatorService.php
│   │   ├── OvertimeCalculatorService.php
│   │   ├── PensionService.php
│   │   ├── SeveranceService.php
│   │   ├── PayslipPdfService.php
│   │   └── FinanceClientService.php
│   └── Leave/
│       └── LeaveAccrualService.php
├── Events/
│   ├── EmployeeCreated.php
│   ├── EmployeeUpdated.php
│   ├── EmployeeArchived.php
│   ├── PayrollRunApproved.php
│   ├── SeveranceCalculated.php
│   └── LeaveApproved.php
├── Listeners/
│   ├── InitialiseEmployeeLeaveBalances.php
│   ├── CloseEmployeeLeaveBalances.php
│   └── PublishLifecycleEventToOutbox.php
├── Jobs/
│   ├── PublishOutboxEvents.php
│   └── PurgeIdempotencyKeys.php
├── app/Console/Commands/
│   └── RunLeaveAccruals.php
├── database/migrations/    (22 tables – see Section 3)
├── routes/api.php
├── config/
└── .env

```

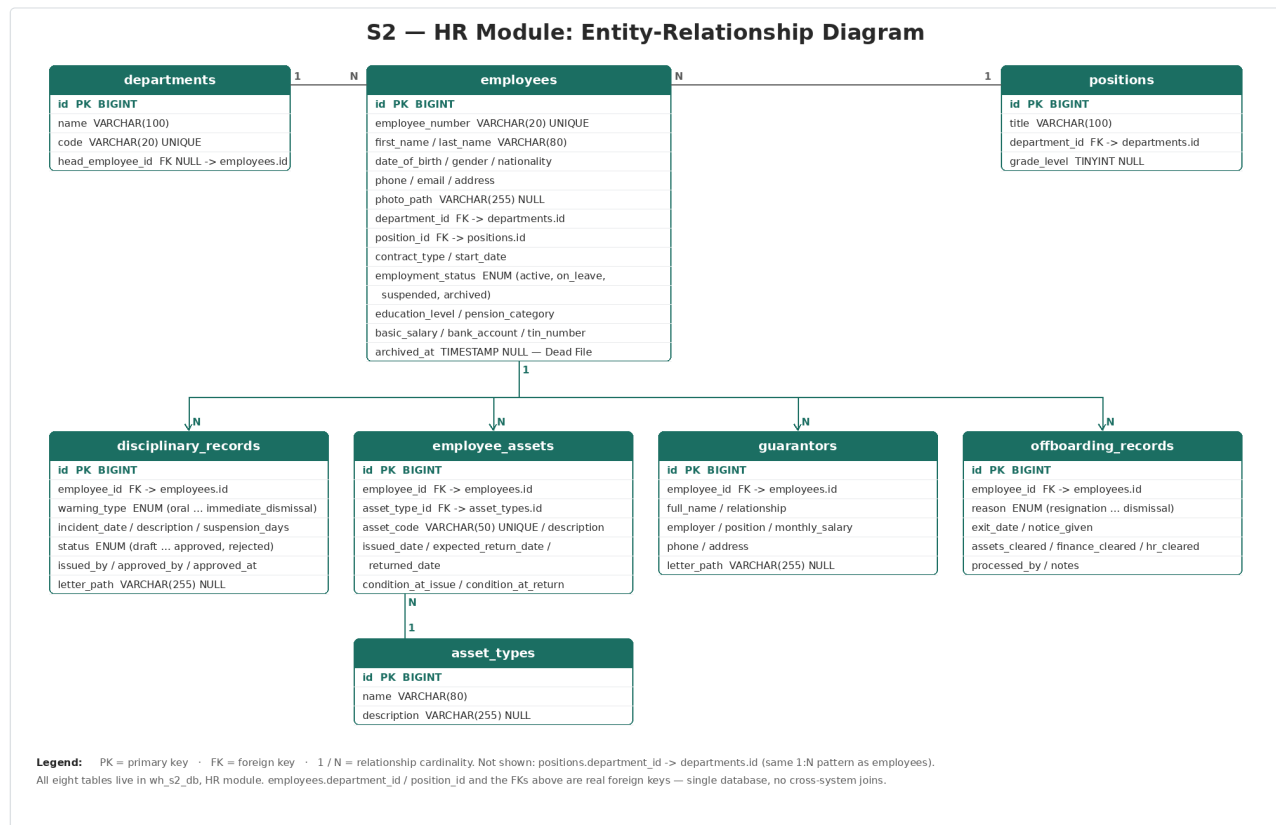
NOTE

`FinanceClientService` is the only outbound HTTP client in S2 — it wraps `POST /api/v1/journal-entries` to S4 (Section 5.1, Section 8.3) with the Service-Key header and Idempotency-Key required by D5. `InitialiseEmployeeLeaveBalances` and `CloseEmployeeLeaveBalances` are in-process event listeners on `EmployeeCreated` / `EmployeeArchived` — they replace what would otherwise be M4 webhook handlers (S0 §5.5).

3. Data Model

S2 owns 22 tables in `wh_s2_db`, grouped into HR (8 tables), Payroll (6 tables), Leave & Attendance (6 tables), and platform plumbing (2 tables). The two ER diagrams below show the HR tables and the Payroll/Leave tables respectively; `employees` appears in both as the hub that Payroll and Leave & Attendance now reference with real foreign keys (ID6).

3.1 employees



Entity-relationship diagram of the eight HR tables in `wh_s2_db`: `departments`, `employees`, `positions`, `disciplinary_records`, `employee_assets`, `asset_types`, `guarantors`, and `offboarding_records`, with `employees` as the central table connected to each of the others by one-to-many relationships.

The `employees` table is the hub of the entire system — Payroll and Leave & Attendance tables hold real foreign keys into it.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_code</code>	VARCHAR(20), UNIQUE	e.g. <code>WH-2024-0317</code>
<code>first_name</code>	VARCHAR(60)	
<code>last_name</code>	VARCHAR(60)	
<code>gender</code>	ENUM(<code>male</code> , <code>female</code>)	
<code>date_of_birth</code>	DATE	
<code>national_id</code>	VARCHAR(40), UNIQUE	

Column	Type	Notes
<code>phone</code>	VARCHAR(20)	
<code>email</code>	VARCHAR(120), UNIQUE NULL	also used as the S1 <code>users.email</code> when an account is created
<code>address</code>	VARCHAR(255) NULL	
<code>photo_path</code>	VARCHAR(255) NULL	path within an object storage bucket (e.g. MinIO/S3-compatible) or local disk — never a direct filesystem path exposed to clients
<code>department_id</code>	BIGINT, FK → <code>departments.id</code>	
<code>position_id</code>	BIGINT, FK → <code>positions.id</code>	
<code>employment_status</code>	ENUM(<code>active</code> , <code>on_leave</code> , <code>suspended</code> , <code>archived</code>)	default <code>active</code> ; drives Section 7 lifecycle
<code>pension_category</code>	ENUM(<code>covered</code> , <code>not_covered</code>)	determines whether <code>PensionService</code> calculates 7%/11% (Section 5.5)
<code>basic_salary</code>	DECIMAL(12,2)	monthly basic salary, ETB
<code>bank_name</code>	VARCHAR(60) NULL	
<code>bank_account_no</code>	VARCHAR(40) NULL	
<code>start_date</code>	DATE	used by <code>LeaveAccrualService</code> (Section 5.7)
<code>archived_at</code>	TIMESTAMP NULL	set when <code>employment_status</code> becomes <code>archived</code> ; never hard-deleted (D13)
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

NOTE

Whenever an `employees` row is created, updated, or archived, S2 writes a corresponding `wh.events.s2.employee.*` row to `event_outbox` (Section 8.1) so S1 can keep its `users` table in sync (ID1, ID2).

3.2 departments

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>name</code>	VARCHAR(80), UNIQUE	e.g. “Front Office”, “Housekeeping”, “Kitchen”
<code>head_employee_id</code>	BIGINT, FK → <code>employees.id</code> , NULL	the department head — used for leave-approval routing (Section 5.8)
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

3.3 positions

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>title</code>	VARCHAR(80)	e.g. “Front Desk Agent”, “Sous Chef”
<code>department_id</code>	BIGINT, FK → <code>departments.id</code>	
<code>grade</code>	VARCHAR(10) NULL	salary grade reference, informational
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

3.4 disciplinary_records

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>action_type</code>	ENUM(<code>oral_warning</code> , <code>first_written_warning</code> , <code>final_written_warning</code> , <code>suspension</code> , <code>termination</code> , <code>immediate_dismissal</code>)	see Section 9.1 escalation table
<code>reason</code>	TEXT	
<code>effective_date</code>	DATE	
<code>suspension_days</code>	SMALLINT NULL	only populated when <code>action_type = suspension</code>
<code>issued_by</code>	BIGINT, FK → <code>employees.id</code>	the manager/HR officer who issued the action
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

3.5 employee_assets

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>asset_type_id</code>	BIGINT, FK → <code>asset_types.id</code>	
<code>serial_number</code>	VARCHAR(80) NULL	
<code>assigned_date</code>	DATE	
<code>returned_date</code>	DATE NULL	set during offboarding clearance (Section 9.2)
<code>condition_on_return</code>	VARCHAR(255) NULL	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

3.6 asset_types

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>name</code>	VARCHAR(80)	e.g. “Laptop”, “Uniform”, “Access Card”, “Mobile Phone”
<code>description</code>	VARCHAR(255) NULL	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

NOTE

`asset_types` is a small lookup table added during the S2 design to give `employee_assets.asset_type_id` a proper catalog rather than a free-text field — it appears in the HR ER diagram and is managed through `AssetController`'s `asset-types` sub-resource (Section 4.1).

3.7 guarantors

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>full_name</code>	VARCHAR(120)	
<code>national_id</code>	VARCHAR(40)	
<code>phone</code>	VARCHAR(20)	
<code>address</code>	VARCHAR(255)	
<code>relationship</code>	VARCHAR(60) NULL	e.g. “Brother”, “Former Employer”
<code>letter_path</code>	VARCHAR(255) NULL	generated bilingual (Amharic/English) PDF letter, Section 9.3
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

3.8 offboarding_records

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>initiated_date</code>	DATE	
<code>reason</code>	ENUM(<code>resignation</code> , <code>termination</code> , <code>retirement</code> , <code>end_of_contract</code> , <code>death</code>)	
<code>last_working_day</code>	DATE	
<code>clearance_status</code>	ENUM(<code>pending</code> , <code>in_progress</code> , <code>completed</code>)	tracks asset-return and guarantor sign-off (Section 9.4)
<code>severance_amount</code>	DECIMAL(12,2) NULL	

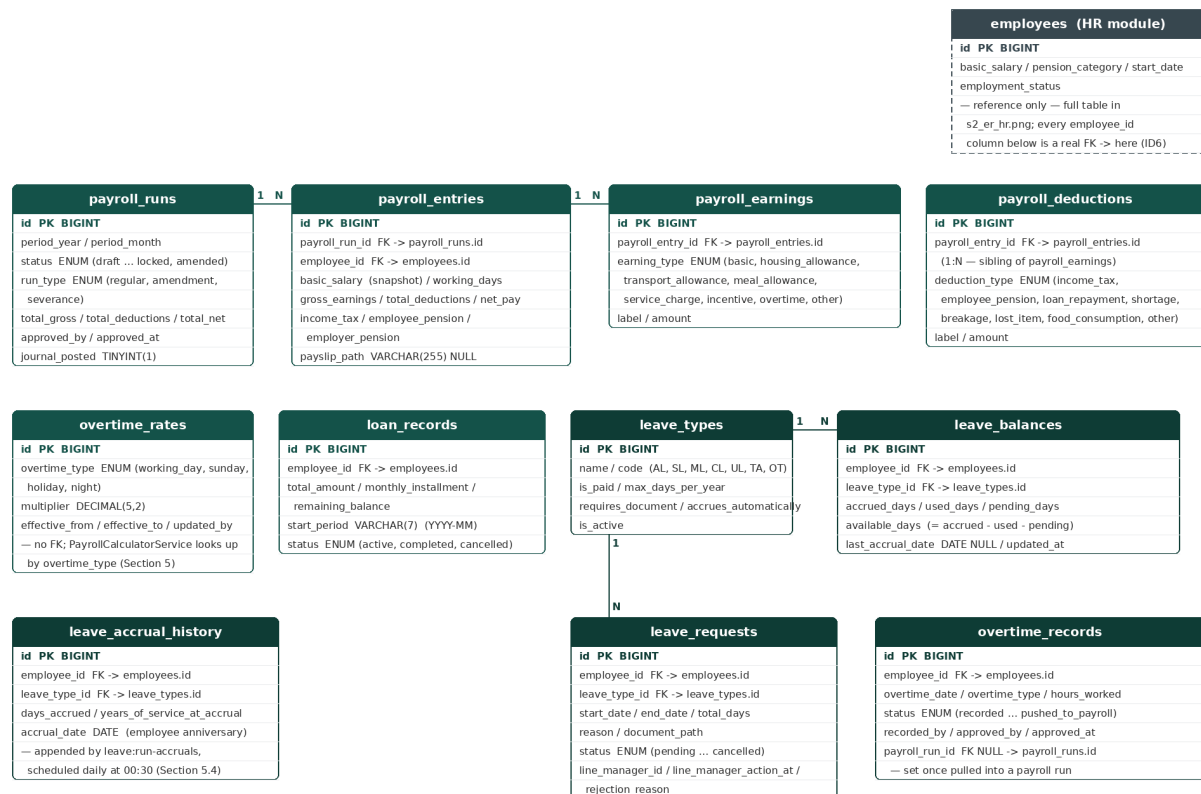
Column	Type	Notes
		new column — written by <code>SeveranceService</code> in the same transaction that calculates severance (Section 5.6, S0 §5.5); NULL until calculated
<code>notes</code>	TEXT NULL	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

NOTE

`offboarding_records`, once created, are **never deleted** — they form part of the employee's permanent “Dead File” (D13, Section 9.5), even after `employees.employment_status` becomes `archived`.

3.9 Payroll & Leave tables

S2 — Payroll & Leave Modules: Entity-Relationship Diagram



Legend: PK = primary key · FK = foreign key · 1 / N = relationship cardinality · dashed border = reference table (full definition in s2_er_hr.png).

Medium teal = Payroll module · light teal = Leave & Attendance module. Every employee_id column above is a real foreign key to employees.id — single database, no cross-system joins (ID6).

Entity-relationship diagram of the payroll and leave/attendance tables in `wh_s2_db`. The employees table is shown as a dashed reference card. Solid foreign keys connect employees to payroll_runs, payroll_entries, payroll_earnings, payroll_deductions, overtime_rates, loan_records, leave_balances, leave_requests, overtime_records, leave_accrual_history, and attendance_records. leave_types is a standalone lookup table referenced by leave_balances and leave_requests. overtime_records includes a nullable foreign key to payroll_runs, indicating which payroll run paid out an approved overtime record.

payroll_runs

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>period_month</code>	TINYINT	1-12
<code>period_year</code>	SMALLINT	
<code>status</code>	ENUM(<code>draft</code> , <code>pending_approval</code> , <code>approved</code> , <code>locked</code>)	see Section 7 lifecycle
<code>generated_by</code>	BIGINT, FK → <code>employees.id</code>	
<code>approved_by</code>	BIGINT, FK → <code>employees.id</code> , NULL	
<code>approved_at</code>	TIMESTAMP NULL	
<code>total_gross</code>	DECIMAL(14,2) NULL	computed when run is generated
<code>total_net</code>	DECIMAL(14,2) NULL	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

payroll_entries

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>payroll_run_id</code>	BIGINT, FK → <code>payroll_runs.id</code>	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>gross_pay</code>	DECIMAL(12,2)	sum of <code>payroll_earnings</code> for this entry
<code>total_deductions</code>	DECIMAL(12,2)	sum of <code>payroll_deductions</code> for this entry
<code>net_pay</code>	DECIMAL(12,2)	<code>gross_pay</code> - <code>total_deductions</code>
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

payroll_earnings

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>payroll_entry_id</code>	BIGINT, FK → <code>payroll_entries.id</code>	
<code>component</code>	ENUM(<code>basic_salary</code> , <code>transport_allowance</code> , <code>housing_allowance</code> , <code>overtime_pay</code> , <code>other</code>)	see Section 5.2
<code>amount</code>	DECIMAL(12,2)	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

payroll_deductions

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>payroll_entry_id</code>	BIGINT, FK → <code>payroll_entries.id</code>	
<code>component</code>	ENUM(<code>income_tax</code> , <code>pension_employee</code> , <code>loan_repayment</code> , <code>staff_meal</code> , <code>other</code>)	see Section 5.3
<code>amount</code>	DECIMAL(12,2)	
<code>source</code>	ENUM(<code>internal</code> , <code>s3_meal_deduction</code>)	<code>s3_meal_deduction</code> rows arrive via ID12 (Section 4.2)
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

overtime_rates

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>category</code>	ENUM(<code>working_day</code> , <code>sunday</code> , <code>holiday</code> , <code>night</code>)	
<code>multiplier</code>	DECIMAL(4,2)	e.g. <code>1.25</code> , <code>2.00</code> , <code>2.50</code> — see Section 5.4
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

loan_records

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>principal_amount</code>	DECIMAL(12,2)	
<code>monthly_repayment</code>	DECIMAL(12,2)	
<code>remaining_balance</code>	DECIMAL(12,2)	decremented by <code>loan_repayment</code> deductions each payroll run
<code>status</code>	ENUM(<code>active</code> , <code>completed</code> , <code>written_off</code>)	
<code>disbursed_at</code>	DATE	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

NOTE

Disbursing a loan posts `Dr 1102 Staff Loans Receivable / Cr 1001 Cash (or 1002 Bank)` to S4 (S0 \$8.2). `POST /employees/{id}/loans` is Idempotency-Key protected (D5).

leave_types

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>code</code>	VARCHAR(10), UNIQUE	<code>AL</code> , <code>SL</code> , <code>ML</code> , <code>CL</code> , <code>UL</code> , <code>TA</code> , <code>OT</code>
<code>name</code>	VARCHAR(60)	e.g. “Annual Leave”, “Maternity Leave”
<code>max_days_per_year</code>	SMALLINT NULL	NULL where not capped (e.g. ML = 90 days per Article 88)
<code>paid</code>	BOOLEAN	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

leave_balances

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>leave_type_id</code>	BIGINT, FK → <code>leave_types.id</code>	
<code>year</code>	SMALLINT	
<code>days_accrued</code>	DECIMAL(5,2)	
<code>days_used</code>	DECIMAL(5,2)	
<code>days_remaining</code>	DECIMAL(5,2)	<code>days_accrued - days_used</code>
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

leave_requests

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>leave_type_id</code>	BIGINT, FK → <code>leave_types.id</code>	
<code>start_date</code>	DATE	
<code>end_date</code>	DATE	
<code>days_requested</code>	DECIMAL(5,2)	weekends excluded if <code>EXCLUDE_WEEKENDS_FROM_LEAVE=true</code>
<code>status</code>	ENUM(<code>pending</code> , <code>approved</code> , <code>rejected</code> , <code>cancelled</code>)	
<code>approved_by</code>	BIGINT, FK → <code>employees.id</code> , NULL	the department head or HR manager who approved (Section 5.8)
<code>reason</code>	TEXT NULL	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

NOTE

An `approved` leave request for `AL`, `SL`, or `ML` types triggers `employment_status = on_leave` for the employee (Section 7) and emits `wh.events.s2.leave.approved` to S4 for reporting (Section 8.1).

overtime_records

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>work_date</code>	DATE	
<code>hours</code>	DECIMAL(4,2)	
<code>category</code>	ENUM(<code>working_day</code> , <code>sunday</code> , <code>holiday</code> , <code>night</code>)	matches <code>overtime_rates.category</code>
<code>status</code>	ENUM(<code>pending</code> , <code>approved</code> , <code>paid</code>)	
<code>payroll_run_id</code>	BIGINT, FK → <code>payroll_runs.id</code> , NULL	set to the run that paid this record out (ID6 — real same-database FK)
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

leave_accrual_history

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>leave_type_id</code>	BIGINT, FK → <code>leave_types.id</code>	
<code>year</code>	SMALLINT	
<code>days_accrued</code>	DECIMAL(5,2)	the amount added by that year's accrual run (Section 5.7)
<code>accrual_date</code>	DATE	
<code>created_at</code>	TIMESTAMP	

attendance_records

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>employee_id</code>	BIGINT, FK → <code>employees.id</code>	
<code>attendance_date</code>	DATE	
<code>check_in_time</code>	TIME NULL	
<code>check_out_time</code>	TIME NULL	
<code>source</code>	ENUM(<code>manual</code> , <code>biometric</code>)	how the record was captured
<code>recorded_by</code>	BIGINT, FK → <code>employees.id</code> , NULL	

Column	Type	Notes
		the HR/supervisor user who entered a manual record; NULL for biometric imports
<code>notes</code>	VARCHAR(255) NULL	
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	

NOTE

`attendance_records` did not exist in the original M4 schema; it is introduced in S2 because (a) the architecture diagram's Leave & Attendance module owns an `AttendanceController`, and (b) S4 reads `GET / api/v1/attendance-records` for BI dashboards (S0 §5.3). Attendance data also feeds `OvertimeCalculatorService` (Section 5.4) when computing hours worked beyond the standard shift.

3.10 Platform plumbing tables

event_outbox

Same structure as S1's `event_outbox` (D2, ID10) — every cross-system event S2 emits (Section 8.1) is written here first, then published by the `PublishOutboxEvents` job.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>event_name</code>	VARCHAR(100)	e.g. <code>wh.events.s2.employee.created</code>
<code>payload</code>	JSON	
<code>status</code>	ENUM(<code>pending</code> , <code>published</code> , <code>failed</code>)	
<code>attempts</code>	SMALLINT	
<code>created_at</code> / <code>published_at</code>	TIMESTAMP	

idempotency_keys

New table, required by D5 because S2 is both a caller (loans, payroll approval) and a callee (S3's deduction push) of idempotent endpoints.

Column	Type	Notes
<code>id</code>	BIGINT, PK	
<code>idempotency_key</code>	VARCHAR(80), UNIQUE	client-supplied <code>Idempotency-Key</code> header value
<code>endpoint</code>	VARCHAR(120)	e.g. <code>POST /api/v1/payroll-runs/{id}/approve</code>
<code>request_hash</code>	VARCHAR(64)	SHA-256 of the normalised request body — a replay with a different body for the same key is rejected (409)
<code>response_body</code>	JSON	cached response, replayed verbatim on retry
<code>status_code</code>	SMALLINT	
<code>created_at</code>	TIMESTAMP	
<code>expires_at</code>	TIMESTAMP	purged daily by <code>PurgeIdempotencyKeys</code> (D5)

4. API Endpoint Catalog

All endpoints below are prefixed with `/api/v1` and reached through the S1 gateway (S0 §3). Every request must carry a verified JWT; the **Permission** column lists the `permissions[]` claim required by `CheckPermission` (Section 6 has the full catalog with role grants).

4.1 HR module

Method & Path	Permission	Description
GET <code>/employees</code>	<code>S2.hr.employees.read</code>	Paginated employee list, filterable by department, status
POST <code>/employees</code>	<code>S2.hr.employees.write</code>	Create employee; emits <code>employee.created</code>
GET <code>/employees/{id}</code>	<code>S2.hr.employees.read</code>	Employee detail
PUT <code>/employees/{id}</code>	<code>S2.hr.employees.write</code>	Update employee; emits <code>employee.updated</code>
DELETE <code>/employees/{id}</code>	<code>S2.hr.employees.write</code>	Not a hard delete — see <code>OffboardingController</code>
GET <code>/departments</code>	<code>S2.hr.departments.read</code>	apiResource
POST <code>/departments</code>	<code>S2.hr.departments.write</code>	apiResource
GET <code>/departments/{id}</code>	<code>S2.hr.departments.read</code>	apiResource
PUT <code>/departments/{id}</code>	<code>S2.hr.departments.write</code>	apiResource
DELETE <code>/departments/{id}</code>	<code>S2.hr.departments.write</code>	apiResource
GET <code>/positions</code>	<code>S2.hr.positions.read</code>	apiResource
POST <code>/positions</code>	<code>S2.hr.positions.write</code>	apiResource
GET <code>/positions/{id}</code>	<code>S2.hr.positions.read</code>	apiResource
PUT <code>/positions/{id}</code>	<code>S2.hr.positions.write</code>	apiResource
DELETE <code>/positions/{id}</code>	<code>S2.hr.positions.write</code>	apiResource
GET <code>/employees/{id}/disciplinary-records</code>	<code>S2.hr.disciplinary.read</code>	
POST <code>/employees/{id}/disciplinary-records</code>	<code>S2.hr.disciplinary.write</code>	See Section 9.1 for escalation rules
GET <code>/employees/{id}/assets</code>	<code>S2.hr.assets.read</code>	
POST <code>/employees/{id}/assets</code>	<code>S2.hr.assets.write</code>	Assign an asset
PUT <code>/assets/{id}/return</code>	<code>S2.hr.assets.write</code>	Record return during offboarding clearance
GET <code>/asset-types</code>	<code>S2.hr.assets.read</code>	apiResource
POST <code>/asset-types</code>	<code>S2.hr.assets.write</code>	apiResource
PUT <code>/asset-types/{id}</code>	<code>S2.hr.assets.write</code>	apiResource
DELETE <code>/asset-types/{id}</code>	<code>S2.hr.assets.write</code>	apiResource

Method & Path	Permission	Description
GET /employees/{id}/guarantors	S2.hr.guarantors.read	
POST /employees/{id}/guarantors	S2.hr.guarantors.write	Generates bilingual PDF letter (Section 9.3)
GET /offboarding-records	S2.hr.offboarding.read	The Dead File index
POST /employees/{id}/offboarding	S2.hr.offboarding.write	Initiates offboarding (Section 9.4); may trigger <code>SeveranceService</code>
PUT /offboarding-records/{id}	S2.hr.offboarding.write	Update clearance status

4.2 Payroll module

Method & Path	Permission	Description
GET /payroll-runs	S2.payroll.runs.read	List runs
POST /payroll-runs	S2.payroll.runs.execute	Generate a draft run for a period (Section 5.1)
GET /payroll-runs/{id}	S2.payroll.runs.read	Run detail with totals
GET /payroll-runs/{id}/entries	S2.payroll.runs.read	Per-employee entries
POST /payroll-runs/{id}/submit	S2.payroll.runs.execute	<code>draft</code> → <code>pending_approval</code>
POST /payroll-runs/{id}/approve	S2.payroll.runs.approve	<code>pending_approval</code> → <code>approved</code> ; Idempotency-Key required (D5) ; posts journal entry to S4
POST /payroll-runs/{id}/lock	S2.payroll.runs.approve	<code>approved</code> → <code>locked</code> (D13 — immutable thereafter)
GET /employees/{id}/payslip/{runId}	S2.payroll.payslips.read	Renders PDF payslip
GET /overtime-rates	S2.payroll.rates.read	apiResource
PUT /overtime-rates/{id}	S2.payroll.rates.write	apiResource
GET /employees/{id}/loans	S2.payroll.loans.read	
POST /employees/{id}/loans	S2.payroll.loans.write	Idempotency-Key required (D5) ; posts disbursement journal entry to S4
POST /employees/{id}/deductions	(Service-Key, see note)	Inbound from S3 — ID12. Records a <code>staff_meal</code> deduction for the employee's next payroll entry. Idempotency-Key required (D5)

NOTE

`POST /employees/{id}/deductions` is the one retained S2 ↔ S3 business call (ID12). It is authenticated with the platform Service-Key (S0 §4, not a user JWT), so it has no `permissions[]` claim —

`ServiceKeyAuthenticate` middleware verifies the caller is S3 instead. The request must carry an `Idempotency-Key` header; replays with the same key return the cached response (D5).

4.3 Leave & Attendance module

Method & Path	Permission	Description
<code>GET /leave-types</code>	<code>S2.leave.types.read</code>	apiResource
<code>POST /leave-types</code>	<code>S2.leave.types.write</code>	apiResource
<code>PUT /leave-types/{id}</code>	<code>S2.leave.types.write</code>	apiResource
<code>GET /employees/{id}/leave-balances</code>	<code>S2.leave.balances.read</code>	
<code>GET /leave-requests</code>	<code>S2.leave.requests.read</code>	List, filterable by status/employee/department
<code>POST /leave-requests</code>	<code>S2.leave.requests.write</code>	Employee/HR creates a request
<code>POST /leave-requests/{id}/approve</code>	<code>S2.leave.approve</code>	Department head or HR manager (Section 5.8)
<code>POST /leave-requests/{id}/reject</code>	<code>S2.leave.approve</code>	
<code>POST /leave-requests/{id}/cancel</code>	<code>S2.leave.requests.write</code>	
<code>GET /employees/{id}/overtime-records</code>	<code>S2.leave.overtime.read</code>	
<code>POST /employees/{id}/overtime-records</code>	<code>S2.leave.overtime.write</code>	Record worked overtime
<code>POST /overtime-records/{id}/approve</code>	<code>S2.leave.overtime.approve</code>	Required before a record can be paid in a payroll run
<code>GET /employees/{id}/attendance</code>	<code>S2.leave.attendance.read</code>	
<code>POST /attendance</code>	<code>S2.leave.attendance.write</code>	Manual entry; biometric imports use the same endpoint with <code>source=biometric</code>

CONVENTION

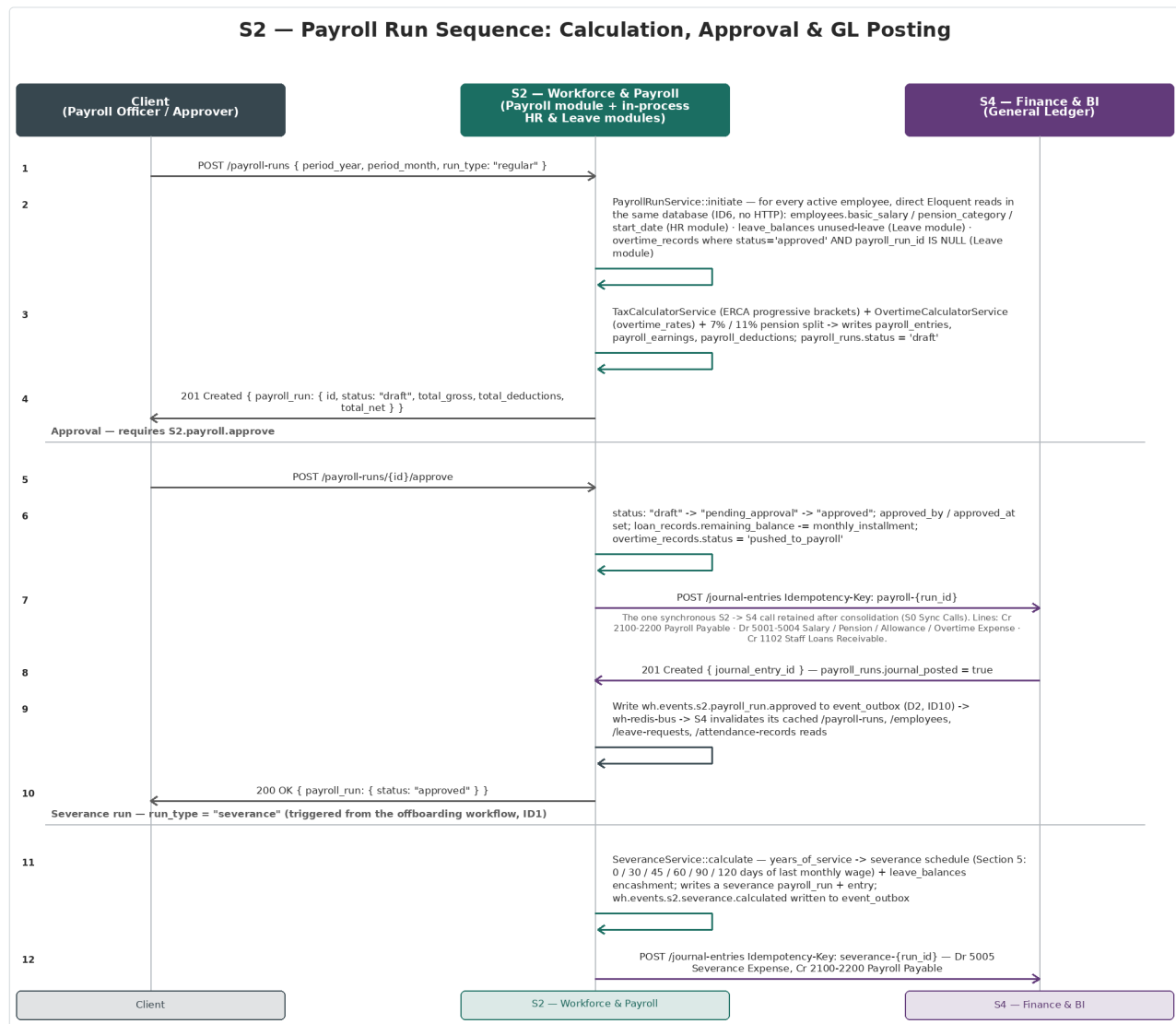
Removed endpoints (ID6). The original M4 spec exposed `GET /employees/{id}/unused-leave-balance` and `GET /employees/{id}/overtime-approved` purely so M3 could pull this data over HTTP. In S2, `PayrollRunController` and `SeveranceService` read `leave_balances` and `overtime_records` directly — **these two endpoints are removed** and have no S2 equivalent.

Pagination, errors, rate limits. All list endpoints use `?page= / ?per_page=` cursor-free pagination (D7); error responses follow the standard `{ "error": { "code", "message", "details" } }` envelope (D6); all endpoints are subject to the platform-wide rate limit defined in S0 §10 unless otherwise noted.

5. Core Business Logic

This section describes S2's calculation engines — payroll, tax, overtime, pension, severance, and leave accrual — in prose and tables. None of this logic is shown as code; it is described precisely enough that any developer can implement it directly in `app/Services/Payroll/*` and `app/Services/Leave/*`.

5.1 Payroll run lifecycle overview



Sequence diagram of a payroll run: the payroll officer triggers run generation, S2 reads employee, leave, and overtime data directly via Eloquent (ID6), computes earnings and deductions for each employee, the officer submits and approves the run, S2 posts a journal entry to S4 with an Idempotency-Key, writes a `payroll_run.approved` event to the outbox, and separately the offboarding flow triggers `SeveranceService` which calculates severance, writes it to `offboarding_records`, and posts a second journal entry to S4.

The numbered steps below correspond to the diagram:

1. The payroll officer calls `POST /payroll-runs` for a given month/year. `PayrollRunController` creates a `payroll_runs` row with `status = draft`.

2. For every active employee, the controller reads `employees.basic_salary` and `employees.pension_category` directly (ID6 — no HTTP), plus that employee's approved-but-unpaid `overtime_records` and current-month `leave_balances`.
3. `OvertimeCalculatorService`, `TaxCalculatorService`, and `PensionService` compute each earning/deduction component (Sections 5.2–5.5) and write `payroll_earnings` / `payroll_deductions` rows, then a `payroll_entries` row with `gross_pay`, `total_deductions`, `net_pay`.
4. Any pending `staff_meal` deductions pushed by S3 (ID12, source `s3_meal_deduction`) are included automatically — they were written to `payroll_deductions` ahead of time and are simply picked up here.
5. Any active `loan_records` produce a `loan_repayment` deduction equal to `monthly_repayment` (or the remaining balance if smaller), and `loan_records.remaining_balance` is decremented.
6. `payroll_runs.total_gross` / `total_net` are computed and `status` is set to `pending_approval` via `POST /payroll-runs/{id}/submit`.
7. On `POST /payroll-runs/{id}/approve` (Idempotency-Key required), `status` becomes `approved`, and `FinanceClientService` calls S4's `POST /api/v1/journal-entries` with Idempotency-Key `payroll-{run_id}` and a description summarising: “Cr 2100-2200 Payroll Payable · Dr 5001-5004 Salary/Pension/ Allowance/Overtime Expense · Cr 1102 Staff Loans Receivable” (S0 §8.2).
8. `POST /payroll-runs/{id}/lock` sets `status = locked` — from this point the run and its entries are immutable (D13).
9. A `wh.events.s2.payroll_run.approved` row is written to `event_outbox` (dashed in the diagram) for S4's reporting cache invalidation (Section 8.1).
10. Separately, when an employee's offboarding reaches the severance step (Section 5.6, Section 9.4), `SeveranceService` runs.
11. `SeveranceService` reads `employees.basic_salary`, `employees.start_date`, and `leave_balances` directly (ID6) and computes the severance + leave-encashment total, writing it to `offboarding_records.severance_amount` in the same transaction.
12. `FinanceClientService` posts a second journal entry to S4 with Idempotency-Key `severance-{run_id}`, and a `wh.events.s2.severance.calculated` event is written to the outbox.

NOTE

Step 7's approval requires `S2.payroll.runs.approve`. (The sequence diagram's divider labels this informally as “S2.payroll.approve” — the canonical 4-segment permission string per ID9 is `S2.payroll.runs.approve`, consistent with `S2.payroll.runs.execute` / `S2.payroll.runs.read` used elsewhere in this catalog.)

5.2 Earnings components

Each `payroll_entries` row is built from one or more `payroll_earnings` rows:

Component	Source	Notes
<code>basic_salary</code>	<code>employees.basic_salary</code>	Always present
<code>transport_allowance</code>	Position-level configuration (flat amount, configured per position)	Optional, depends on position
<code>housing_allowance</code>	Position-level configuration	Optional
<code>overtime_pay</code>	<code>OvertimeCalculatorService</code> output (Section 5.4)	Only present if the employee has approved, unpaid <code>overtime_records</code> for the period
<code>other</code>	Manual adjustment entered by payroll officer before submission	Rare — bonuses, corrections

5.3 Deductions and the ERCA tax schedule

Each `payroll_entries` row also has one or more `payroll_deductions` rows:

Component	Source	Notes
<code>income_tax</code>	<code>TaxCalculatorService</code>	Progressive bracket calculation below
<code>pension_employee</code>	<code>PensionService</code>	7% of basic salary, <code>pension_category = covered</code> only (Section 5.5)
<code>loan_repayment</code>	<code>loan_records.monthly_repayment</code>	Only if the employee has an <code>active</code> loan
<code>staff_meal</code>	Pushed by S3 via <code>POST / employees/{id}/deductions</code> (ID12)	<code>source = s3_meal_deduction</code>
<code>other</code>	Manual adjustment	Rare

ERCA progressive income tax brackets (applied to *taxable income*, defined below):

Monthly Taxable Income (ETB)	Rate	Deduction
0 – 600	0%	0
601 – 1,650	10%	0
1,651 – 3,200	15%	90
3,201 – 5,250	20%	240
5,251 – 7,800	25%	502.50
7,801 – 10,900	30%	892.50
10,901 and above	35%	1,437.50

Formula:

$$\text{Taxable Income} = \text{Gross Pay} - \text{Employee Pension Contribution}$$

$\text{Income Tax} = (\text{Taxable Income} \times \text{Rate}) - \text{Deduction}$, using the bracket that `Taxable Income` falls into.

`TaxCalculatorService` finds the matching bracket, applies the formula, and writes the result as the `income_tax` deduction.

5.4 Overtime pay

`overtime_rates` holds the four multiplier categories:

Category	Multiplier
<code>working_day</code>	1.25×
<code>sunday</code>	2.00×
<code>holiday</code>	2.50×
<code>night</code>	1.25×

Formula:

$\text{Hourly Rate} = \text{Basic Salary} \div (\text{WORKING_DAYS_PER_MONTH} \times \text{WORKING_HOURS_PER_DAY})$

$\text{Overtime Pay} = \text{Hourly Rate} \times \text{Multiplier} \times \text{Hours Worked}$

With `WORKING_DAYS_PER_MONTH = 26` and `WORKING_HOURS_PER_DAY = 8` (Section 10.1), `OvertimeCalculatorService` sums this across every `approved overtime_records` row for the employee within the payroll period, writes the total as the `overtime_pay` earning, and sets each consumed record's `status = paid` and `payroll_run_id` to the current run.

5.5 Pension contributions

Pension Category	Employee Rate	Employer Rate	Notes
<code>covered</code>	7% of basic salary	11% of basic salary	Employee rate is deducted (<code>pension_employee</code>); employer rate is informational for S4's expense posting (account 5002) and is not deducted from the employee
<code>not_covered</code>	—	—	No pension deduction or contribution

`PensionService` checks `employees.pension_category` and, if `covered`, writes `pension_employee = basic_salary × 0.07` to `payroll_deductions`. The taxable-income formula in Section 5.3 subtracts this amount before applying the tax brackets.

5.6 Severance calculation

Severance applies when an employee's offboarding reaches the severance step (Section 9.4), for reasons other than `immediate_dismissal` for gross misconduct (per Ethiopian labour law).

Severance schedule (based on completed years of service):

Years of Service	Severance
Less than 1 year	0 days
1 – 2 years	30 days of last monthly wage
3 – 5 years	45 days of last monthly wage
6 – 10 years	60 days of last monthly wage
11 – 20 years	90 days of last monthly wage
More than 20 years	120 days of last monthly wage

Formulas:

`Daily Rate = Basic Salary ÷ 26`

`Severance Pay = Daily Rate × Days (from schedule above, based on employees.start_date)`

`Leave Encashment = Daily Rate × Unused Leave Days` (from `leave_balances.days_remaining` for the `AL` leave type, read directly per ID6)

`Total Severance = Severance Pay + Leave Encashment`

`SeveranceService`:

1. Reads `employees.basic_salary` and `employees.start_date` to determine completed years of service and the daily rate.
2. Reads `leave_balances` for the `AL` leave type to compute leave encashment.
3. Writes `Total Severance` to `offboarding_records.severance_amount` in the same transaction.
4. Emits `wh.events.s2.severance.calculated` to `event_outbox`.
5. `FinanceClientService` posts the corresponding journal entry to S4: `Dr 5005 Severance Expense / Cr 1001 Cash (or 1002 Bank)` (S0 \$8.2).

5.7 Annual leave accrual

Leave Type	Code	Max Days/Year	Paid	Notes
Annual Leave	<code>AL</code>	Accrues per formula below, capped at 30	Yes	
Sick Leave	<code>SL</code>	(policy-defined)	Yes	
Maternity Leave	<code>ML</code>	90	Yes	Per Labour Proclamation Article 88
Casual Leave	<code>CL</code>	(policy-defined)	Yes	
Unpaid Leave	<code>UL</code>	(uncapped)	No	
	<code>TA</code>	(policy-defined)	Yes	

Leave Type	Code	Max Days/Year	Paid	Notes
Travel/Transfer Allowance Leave				
Other Leave	OT	(policy-defined)	Varies	

Annual leave accrual formula:

`Annual Days = min(15 + completed_years_of_service, 30)`

This applies starting from the employee's second year of service (i.e. after the first full year). `LeaveAccrualService`, invoked by the `RunLeaveAccruals` scheduled command at **00:30 daily**, checks each employee's `employees.start_date` (read directly, ID6) against the current date; on an employee's service anniversary, it computes `Annual Days` for the new year, adds a `leave_accrual_history` row, and increments `leave_balances.days_accrued` for the `AL` type accordingly.

NOTE

`InitialiseEmployeeLeaveBalances`, an in-process listener on `EmployeeCreated`, creates the initial `leave_balances` rows for all seven leave types when an employee is hired (S0 \$5.5 — formerly an M2 → M4 webhook). `CloseEmployeeLeaveBalances`, a listener on `EmployeeArchived`, marks balances as closed when an employee leaves.

5.8 Attendance and overtime approval workflow

- Attendance is recorded daily via `POST /attendance`, either manually by a supervisor (`source = manual`, `recorded_by` set) or imported from a biometric device (`source = biometric`).
- When an employee's recorded hours for a day exceed the standard shift (`WORKING_HOURS_PER_DAY = 8`), a supervisor may submit an `overtime_records` row via `POST /employees/{id}/overtime-records` with `status = pending` .
- The employee's department head (`departments.head_employee_id`) or an HR manager approves it via `POST /overtime-records/{id}/approve` (`S2.leave.overtime.approve`), setting `status = approved` .
- Only `approved` overtime records are picked up by `OvertimeCalculatorService` during the next payroll run (Section 5.4), after which they are marked `paid` .
- Leave requests follow the same department-head-or-HR-manager approval pattern via `POST /leave-requests/{id}/approve` (`S2.leave.approve`) — approving an `AL / SL / ML` request transitions the employee to `employment_status = on_leave` (Section 7) and decrements `leave_balances.days_remaining` by `days_requested` .

6. Role-Based Access Control

S2 enforces authorization with the same `CheckPermission` middleware described in S0 §4 — it reads the `permissions[]` claim from the verified JWT and rejects the request with `403` if the required permission is absent. This section catalogs every S2 permission, grouped by module per ID9's `S2.{domain}.{resource}.{action}` convention.

6.1 Permission catalog

HR domain (`S2.hr.*`)

Permission	Default Roles
<code>S2.hr.employees.read</code>	<code>hr_manager</code> , <code>department_head</code> (own department only), <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S2.hr.employees.write</code>	<code>hr_manager</code> , <code>super_admin</code>
<code>S2.hr.departments.read</code>	<code>hr_manager</code> , <code>department_head</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S2.hr.departments.write</code>	<code>hr_manager</code> , <code>super_admin</code>
<code>S2.hr.positions.read</code>	<code>hr_manager</code> , <code>department_head</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S2.hr.positions.write</code>	<code>hr_manager</code> , <code>super_admin</code>
<code>S2.hr.disciplinary.read</code>	<code>hr_manager</code> , <code>department_head</code> (own department only), <code>general_manager</code> , <code>super_admin</code>
<code>S2.hr.disciplinary.write</code>	<code>hr_manager</code> , <code>super_admin</code>
<code>S2.hr.assets.read</code>	<code>hr_manager</code> , <code>general_manager</code> , <code>super_admin</code>
<code>S2.hr.assets.write</code>	<code>hr_manager</code> , <code>super_admin</code>
<code>S2.hr.guarantors.read</code>	<code>hr_manager</code> , <code>super_admin</code>
<code>S2.hr.guarantors.write</code>	<code>hr_manager</code> , <code>super_admin</code>
<code>S2.hr.offboarding.read</code>	<code>hr_manager</code> , <code>general_manager</code> , <code>super_admin</code>
<code>S2.hr.offboarding.write</code>	<code>hr_manager</code> , <code>super_admin</code>

Payroll domain (`S2.payroll.*`)

Permission	Default Roles
<code>S2.payroll.runs.read</code>	<code>payroll_officer</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S2.payroll.runs.execute</code>	<code>payroll_officer</code> , <code>super_admin</code>
<code>S2.payroll.runs.approve</code>	<code>payroll_officer</code> , <code>super_admin</code>
<code>S2.payroll.payslips.read</code>	<code>payroll_officer</code> , <code>super_admin</code>
<code>S2.payroll.rates.read</code>	<code>payroll_officer</code> , <code>general_manager</code> , <code>super_admin</code>

Permission	Default Roles
<code>S2.payroll.rates.write</code>	<code>payroll_officer</code> , <code>super_admin</code>
<code>S2.payroll.loans.read</code>	<code>payroll_officer</code> , <code>general_manager</code> , <code>super_admin</code>
<code>S2.payroll.loans.write</code>	<code>payroll_officer</code> , <code>super_admin</code>

Leave & Attendance domain (`S2.leave.*`)

Permission	Default Roles
<code>S2.leave.types.read</code>	<code>hr_manager</code> , <code>department_head</code> , <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S2.leave.types.write</code>	<code>hr_manager</code> , <code>super_admin</code>
<code>S2.leave.balances.read</code>	<code>hr_manager</code> , <code>department_head</code> (own department only), <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S2.leave.requests.read</code>	<code>hr_manager</code> , <code>department_head</code> (own department only), <code>general_manager</code> , <code>report_viewer</code> , <code>super_admin</code>
<code>S2.leave.requests.write</code>	<code>hr_manager</code> , <code>department_head</code> , <code>super_admin</code>
<code>S2.leave.approve</code>	<code>hr_manager</code> , <code>department_head</code> , <code>super_admin</code>
<code>S2.leave.overtime.read</code>	<code>hr_manager</code> , <code>payroll_officer</code> , <code>department_head</code> , <code>general_manager</code> , <code>super_admin</code>
<code>S2.leave.overtime.write</code>	<code>hr_manager</code> , <code>department_head</code> , <code>super_admin</code>
<code>S2.leave.overtime.approve</code>	<code>hr_manager</code> , <code>department_head</code> , <code>super_admin</code>
<code>S2.leave.attendance.read</code>	<code>hr_manager</code> , <code>department_head</code> , <code>general_manager</code> , <code>super_admin</code>
<code>S2.leave.attendance.write</code>	<code>hr_manager</code> , <code>department_head</code> , <code>super_admin</code>

6.2 System roles — S2 access summary

This table mirrors S0 §7.2's Role → System Access Matrix, filtered to S2:

Role	S2 Access
<code>hr_manager</code>	Full — all HR and Leave & Attendance permissions
<code>payroll_officer</code>	Full — all Payroll permissions
<code>department_head</code>	Leave approval (<code>S2.leave.approve</code> , <code>S2.leave.overtime.approve</code>) plus read access scoped to their own department's employees, leave, and attendance
<code>general_manager</code>	Read — all <code>*.read</code> permissions across HR, Payroll, and Leave
<code>report_viewer</code>	Read — <code>*.read</code> permissions for employees, departments, positions, payroll runs, leave types/balances/requests (no payslip or loan detail)

Role	S2 Access
<code>super_admin</code>	Full — every permission listed above
<code>finance_manager</code>	— (no S2 access; see decision box below)
<code>accountant</code>	— (no S2 access; see decision box below)

DESIGN DECISION

Permission renaming and the removal of `finance_manager` / `accountant` access to S2.

- Renaming (ID9).** The original catalogs used `M2.*`, `M3.*`, `M4.*` (e.g. `M3.payroll.approve`, `M2.offboard`). Under the unified `S{n}.{domain}.{resource}.{action}` convention, these become `S2.hr.*`, `S2.payroll.*`, and `S2.leave.*` as catalogued above. Anywhere a former permission string appears in legacy documentation, treat it as superseded by its S2 equivalent in this table.
- `finance_manager` and `accountant` lose direct S2 access.** In the original M3 catalog, `finance_manager` and `accountant` held `M3.payroll.approve` / `M3.payroll.read` so they could review payroll for GL purposes. Under S0 §7.2, **that access is removed** — these roles now have **no S2 permissions at all**. Their payroll visibility is satisfied entirely by S4: payroll-run totals and severance amounts arrive at S4 as journal entries (Section 5.1, Section 8.3) and as cached reads of S2's payroll-run data (S0 §5.3–5.4, 300s TTL). If a developer finds legacy seed data granting `finance_manager` / `accountant` an `S2.payroll.*` permission, it should be removed during the S1 permission-seeding migration (Section 6.5).

This mirrors the equivalent decision in S1 regarding `hr_manager` losing direct S1 access — in both cases, the rationale is the same: **cross-system data should be consumed through S4's reporting layer, not through direct grants on another system's permissions.**

6.3 CheckPermission middleware contract

`CheckPermission` behaves identically to the contract described in S0 §4 and implemented in S1:

- Reads `permissions[]` from the verified JWT (already validated by `VerifyJwt` upstream at the gateway).
- Route definitions in `routes/api.php` declare the required permission via middleware parameter, e.g. `->middleware('permission:S2.payroll.runs.approve')`.
- Missing permission → `403 Forbidden` with the standard error envelope (D6).
- Department-scoped permissions (marked “own department only” above) additionally compare the JWT's `employee_id` claim against the target resource's `department_id / employee_id` — implemented as a second check inside the relevant controller method, not in the middleware itself, since the scoping rule differs per resource.

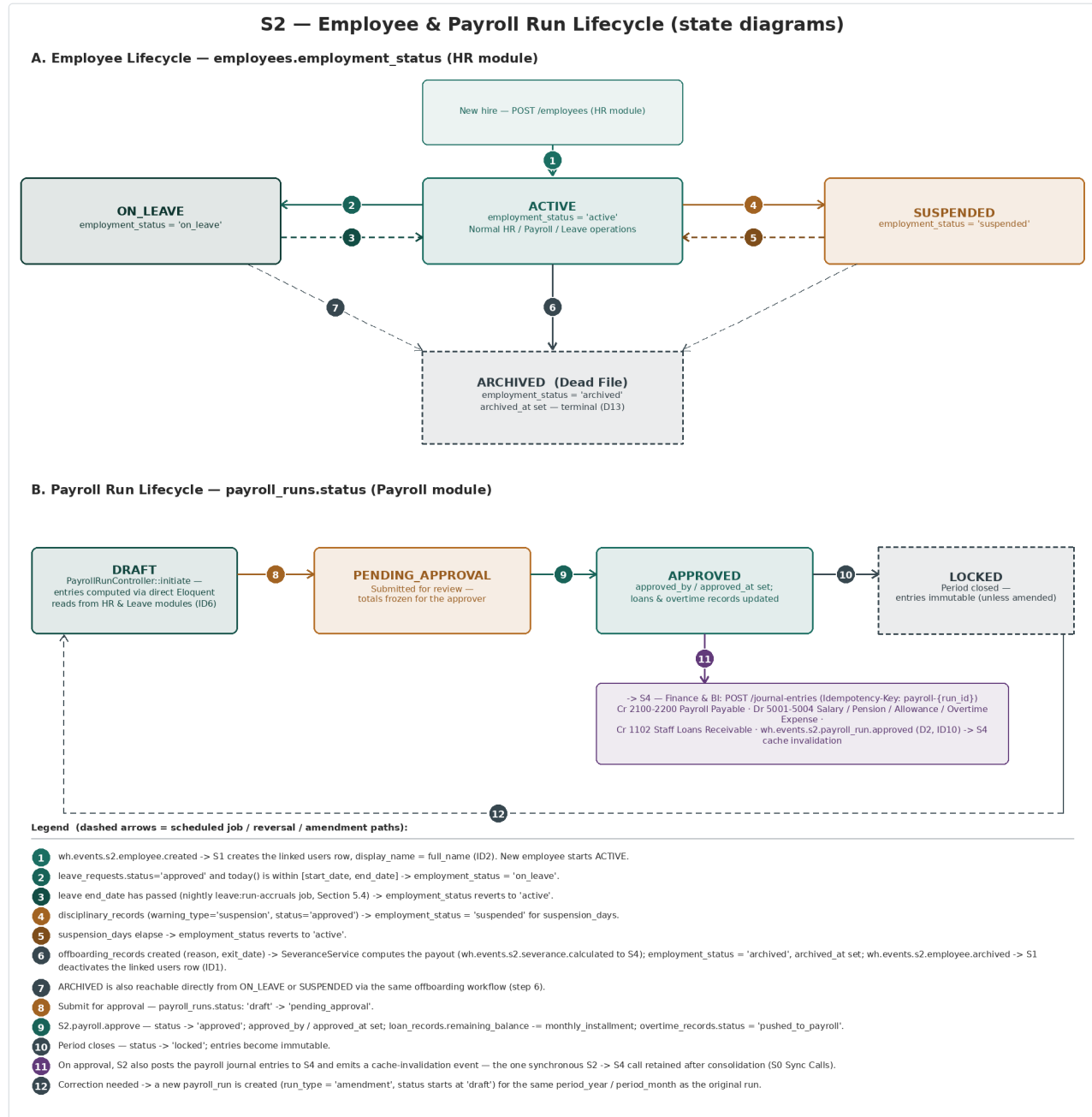
6.4 Permission seeding

CROSS-REFERENCE

The master list of all platform permissions — including every `S2.*` permission in this catalog — is seeded into S1's `permissions` table and assigned to roles via S1's `role_permissions` table, as described in **S0 §7.1** and **S1 §6.4 (Permission Seeding)**. S2 does not store permissions or roles itself; it only reads the `permissions[]` claim from the JWT.

7. Employee & Payroll Run Lifecycle

S2 manages two independent state machines: the **employee lifecycle** (`employees.employment_status`) and the **payroll run lifecycle** (`payroll_runs.status`). The diagram below shows both.



State diagram of System 2's two lifecycles. Section A shows the employee lifecycle with states ACTIVE, ON_LEAVE, SUSPENDED, and ARCHIVED, connected by seven numbered transitions: employee creation into ACTIVE, leave approval into ON_LEAVE and return from leave back to ACTIVE, disciplinary suspension into SUSPENDED and reinstatement back to ACTIVE, and offboarding into the terminal ARCHIVED state which triggers an `employee.archived` event to S1 and creation of a Dead File offboarding record. Section B shows the payroll run lifecycle with states DRAFT, PENDING_APPROVAL, APPROVED, and LOCKED, connected by five numbered transitions: run generation into DRAFT, submission into PENDING_APPROVAL, approval into APPROVED which posts a journal entry to the S4 general ledger, locking into the terminal LOCKED state, and an amendment loop returning PENDING_APPROVAL back to DRAFT for corrections before approval. A twelve-item legend below both sections explains each numbered transition and the `employment_status` and `payroll_runs.status` enum values involved.

7.1 Section A — Employee lifecycle (`employees.employment_status`)

State	Meaning	Entered via
<code>active</code>	Normal working employee	(1) Default status on <code>POST /employees</code>
<code>on_leave</code>	Currently on approved AL/SL/ML leave	(2) <code>POST /leave-requests/{id}/approve</code> for AL/SL/ML; returns to <code>active</code> via (3) when the leave period ends
<code>suspended</code>	Under a disciplinary suspension	(4) <code>POST /employees/{id}/disciplinary-records</code> with <code>action_type = suspension</code> ; returns to <code>active</code> via (5) when <code>suspension_days</code> elapses
<code>archived</code>	No longer employed — terminal	(6) <code>POST /employees/{id}/offboarding</code> on <code>last_working_day</code> ; emits <code>wh.events.s2.employee.archived</code> (Section 8.1), which S1 consumes to deactivate the linked <code>users</code> row (ID1)

Transition (7) represents the permanent record left behind: once `archived`, the employee's `offboarding_records` row remains queryable forever as part of the Dead File (D13, Section 9.5) — `archived` employees are never deleted and never transition to any other state.

7.2 Section B — Payroll run lifecycle (`payroll_runs.status`)

State	Meaning	Entered via
<code>draft</code>	Run generated, entries computed, editable	(8) <code>POST /payroll-runs</code>
<code>pending_approval</code>	Submitted for review	(9) <code>POST /payroll-runs/{id}/submit</code>
<code>approved</code>	Approved; journal entry posted to S4	(10) <code>POST /payroll-runs/{id}/approve</code> (Idempotency-Key required, D5)
<code>locked</code>	Immutable, terminal	(11) <code>POST /payroll-runs/{id}/lock</code>

Transition (12) is the **amendment loop**: if a reviewer finds an error while a run is `pending_approval`, it can be returned to `draft` (no dedicated endpoint beyond re-opening via `PUT` on the relevant `payroll_entries` / `payroll_earnings` / `payroll_deductions` rows, which is only permitted while `status` is `draft` or `pending_approval`) for correction before re-submission.

CONVENTION

D13 — immutability of locked runs. Once a `payroll_runs` row reaches `locked`, no `payroll_entries`, `payroll_earnings`, or `payroll_deductions` row belonging to it may be modified or deleted by any endpoint. Corrections to a locked run require a new payroll run (e.g. an adjustment run) in a subsequent period — this preserves the audit trail between S2's payroll records and the journal entries already posted to S4.

8. Event-Driven Integration

8.1 Events emitted by S2

S2 writes the following events to `event_outbox` (Section 8.4); the `PublishOutboxEvents` job publishes them to the Redis-backed event bus described in S0 §5.1.

Event	Consumed by	Payload (key fields)
<code>wh.events.s2.employee.created</code>	S1	<code>employee_id</code> , <code>employee_code</code> , <code>first_name</code> , <code>last_name</code> , <code>email</code> , <code>department_id</code> , <code>position_id</code> , <code>employment_status</code> — S1 creates a matching <code>users</code> row (ID1)
<code>wh.events.s2.employee.updated</code>	S1	<code>employee_id</code> , <code>display_name</code> (derived from <code>first_name + last_name</code>), <code>email</code> , <code>employment_status</code> — S1 keeps <code>users.display_name</code> in sync for the JWT <code>name</code> claim (ID2)
<code>wh.events.s2.employee.archived</code>	S1	<code>employee_id</code> , <code>archived_at</code> — S1 deactivates the linked <code>users</code> row (ID1)
<code>wh.events.s2.payroll_run.approved</code>	S4	<code>payroll_run_id</code> , <code>period_month</code> , <code>period_year</code> , <code>total_gross</code> , <code>total_net</code> , <code>approved_at</code> — S4 invalidates its cached payroll-run reads (S0 §5.4)
<code>wh.events.s2.severance.calculated</code>	S4	<code>employee_id</code> , <code>offboarding_record_id</code> , <code>severance_amount</code> , <code>calculated_at</code> — informational; the journal entry itself is posted synchronously (Section 5.6, step 12)
<code>wh.events.s2.leave.approved</code>	S4	<code>leave_request_id</code> , <code>employee_id</code> , <code>leave_type_code</code> , <code>start_date</code> , <code>end_date</code> , <code>days_requested</code> — feeds S4's leave/attendance reporting dashboards

8.2 Events consumed by S2

S2 consumes no cross-system events. Every interaction that, in the original M2/M3/M4 design, would have been a cross-service event (employee creation initialising leave balances, employee archival closing leave balances, payroll pulling leave/overtime data, severance updating the Dead File) is now an **in-process listener** or **direct Eloquent query** under ID6 and S0 §5.5:

Former cross-service event	Now (in-process)
M2 <code>EmployeeCreated</code> → M4 initialises leave balances	

Former cross-service event	Now (in-process)
	<code>InitialiseEmployeeLeaveBalances</code> listener on the local <code>EmployeeCreated</code> event (Section 2.3)
M2 <code>EmployeeArchived</code> → M4 closes leave balances	<code>CloseEmployeeLeaveBalances</code> listener on the local <code>EmployeeArchived</code> event
M3 → M2 read employee salary/pension/start date	Direct Eloquent query in <code>PayrollRunController</code> and <code>SeveranceService</code> (Section 5.1, 5.6)
M3 → M4 read unused leave balance / approved overtime	Direct Eloquent query in <code>PayrollRunController</code> / <code>SeveranceService</code>
M3 → M2 write severance amount to Dead File	<code>SeveranceService</code> writes <code>offboarding_records.severance_amount</code> in the same transaction (Section 5.6)

S2 still runs the standard `redis-bus-subscriber` worker process required by D2/ID8 platform conventions (so the worker fleet is uniform across all four systems), but it currently has **no event subscriptions registered** — it is idle by design.

8.3 Synchronous calls

Direction	Call	Notes
S2 → S4	<code>POST /api/v1/journal-entries</code>	Used for payroll-run approval (Idempotency-Key <code>payroll-{run_id}</code>) and severance (Idempotency-Key <code>severance-{run_id}</code>), and for loan disbursement (Idempotency-Key from <code>POST /employees/{id}/loans</code>). Authenticated with the platform Service-Key (S0 §4).
S3 → S2	<code>POST /api/v1/employees/{id}/deductions</code>	ID12 — staff meal charges. Inbound; authenticated with the platform Service-Key; Idempotency-Key required (D5, Section 8.5).
S4 → S2	<code>GET /api/v1/employees</code> , <code>/payroll-runs</code> , <code>/leave-requests</code> , <code>/attendance-records</code>	Read-only, cached by S4 with a 300-second TTL (S0 §5.4).

8.4 Outbox mechanics

S2's `event_outbox` table and `PublishOutboxEvents` job follow the same pattern as S1 (D2, ID10):

- Within the same database transaction that writes the business change (e.g. creating an `employees` row), S2 inserts a row into `event_outbox` with `status = pending`.
- `PublishOutboxEvents`, scheduled every **10 seconds**, selects `pending` rows, publishes each to the Redis event bus topic matching `event_name`, and marks it `published`.
- If publishing fails, `attempts` is incremented and the row remains `pending` for retry; after a configured maximum, it is marked `failed` for manual investigation.

This guarantees that an event is never lost due to a crash between the business write and the publish — either both happen (same transaction) or neither does.

8.5 Idempotency mechanics (D5)

The `idempotency_keys` table (Section 3.10) protects three endpoints:

Endpoint	Direction	Behaviour on retry with same key
<code>POST /payroll-runs/{id}/approve</code>	Inbound (payroll officer)	Returns the cached <code>response_body</code> / <code>status_code</code> from the first call; does not re-post the journal entry to S4
<code>POST /employees/{id}/loans</code>	Inbound (payroll officer)	Returns the cached response; does not re-disburse the loan or re-post the journal entry
<code>POST /employees/{id}/deductions</code>	Inbound (from S3, ID12)	Returns the cached response; does not create a duplicate <code>payroll_deductions</code> row

For each, `EnsureIdempotent` middleware: (1) looks up `idempotency_keys` by the `Idempotency-Key` header; (2) if found and `request_hash` matches, replays `response_body` / `status_code`; (3) if found with a different `request_hash`, returns `409 Conflict`; (4) if not found, proceeds and records the response on completion. `PurgeIdempotencyKeys` runs daily and deletes rows past `expires_at`.

9. Disciplinary Actions, Assets, Guarantors & the Dead File Offboarding Workflow

This section covers the HR-module workflows that do not fit neatly into the data-model or API tables above.

9.1 Disciplinary escalation

`disciplinary_records.action_type` follows a progressive escalation ladder. The system does not auto-escalate — each action is recorded individually by HR — but the table below documents the expected progression and any automatic system behaviour triggered by each action:

Action Type	Typical Trigger	System Behaviour
<code>oral_warning</code>	First minor infraction	Record only; no status change
<code>first_written_warning</code>	Repeat or moderate infraction	Record only; no status change
<code>final_written_warning</code>	Continued infraction after written warning	Record only; no status change
<code>suspension</code>	Serious infraction	<code>employees.employment_status</code> → <code>suspended</code> ; <code>suspension_days</code> set; auto-reverts to <code>active</code> when the period elapses (Section 7.1, transition 5)
<code>termination</code>	Cause for dismissal with notice	Initiates the offboarding workflow (Section 9.4) with <code>offboarding_records.reason = termination</code> ; full severance calculation applies (Section 5.6)
<code>immediate_dismissal</code>	Gross misconduct	Initiates offboarding with <code>reason = termination</code> ; severance does not apply per Ethiopian labour law exceptions — <code>SeveranceService</code> is skipped and <code>offboarding_records.severance_amount</code> remains <code>0.00</code>

9.2 Asset assignment and clearance

1. HR assigns a company asset to an employee via `POST /employees/{id}/assets`, referencing an `asset_types` row (e.g. “Laptop”, “Uniform”, “Access Card”) and recording `serial_number` and `assigned_date`.
2. During offboarding (Section 9.4), HR reviews all `employee_assets` rows for the departing employee with `returned_date IS NULL`.
3. Each outstanding asset is returned via `PUT /assets/{id}/return`, which sets `returned_date` and `condition_on_return`.
4. `offboarding_records.clearance_status` cannot move to `completed` while any `employee_assets` row for the employee has `returned_date IS NULL`.

9.3 Guarantor registration

Many positions (particularly those handling cash, inventory, or company property) require a registered guarantor before the employee can be confirmed. `POST /employees/{id}/guarantors`:

1. Records the guarantor's identity (`full_name`, `national_id`, `phone`, `address`, `relationship`).
2. Generates a **bilingual (Amharic/English) PDF letter** confirming the guarantor's responsibility, stored at `guarantors.letter_path`. PDF generation uses the DejaVu font family, which provides full Amharic (Ethiopic script) glyph coverage alongside Latin text — the same font family used for the platform's other generated documents (e.g. payslips, Section 4.2).

9.4 The Dead File offboarding workflow

“Dead File” is the platform's term for an employee's permanent post-employment record — the `offboarding_records` row plus everything it references. The workflow:

1. **Initiation** — `POST /employees/{id}/offboarding` creates an `offboarding_records` row with `reason`, `initiated_date`, and `last_working_day`, and sets `clearance_status = pending`.
2. **Clearance** — HR works through outstanding `employee_assets` returns (Section 9.2) and confirms any `guarantors` obligations are resolved. `clearance_status` moves to `in_progress`, then `completed`.
3. **Severance calculation** — if `reason` is not `immediate_dismissal`, `SeveranceService` runs as described in Section 5.6, writing `offboarding_records.severance_amount` and posting the journal entry to S4.
4. **Archival** — on `last_working_day`, `employees.employment_status` becomes `archived` (Section 7.1, transition 6), and `wh.events.s2.employee.archived` is emitted so S1 deactivates the corresponding `users` row (ID1).
5. **Permanent record** — the `offboarding_records` row, along with the employee's full `employees`, `disciplinary_records`, `employee_assets`, and `guarantors` history, remains queryable indefinitely. None of these rows are ever hard-deleted.

9.5 Dead File retention rule

CONVENTION

D13 — the Dead File is never hard-deleted. `employees`, `offboarding_records`, `disciplinary_records`, `employee_assets`, and `guarantors` rows for archived employees must remain in `wh_s2_db` indefinitely, queryable via `GET /offboarding-records` and `GET /employees/{id}` (which continues to return `archived` employees). This is both a labour-law record-keeping requirement and the audit trail for any severance payment already posted to S4's general ledger (Section 5.6).

10. Security, Secrets & Operational Conventions

10.1 Platform conventions (D1–D14) as applied to S2

ID	Convention	How S2 applies it
D1	One Laravel app, one database per system	<code>wh_s2_db</code> , 22 tables (Section 3)
D2	<code>event_outbox</code> + <code>PublishOutboxEvents</code> (every 10s)	Section 8.4; 6 emitted event types (Section 8.1)
D3	Gateway-issued JWT verified on every request	<code>VerifyJwt</code> middleware (S0 §3–4)
D4	<code>permissions[]</code> claim drives authorization	<code>CheckPermission</code> middleware (Section 6)
D5	<code>idempotency_keys</code> for write-once endpoints	Section 3.10, Section 8.5 — protects payroll approval, loan disbursement, and inbound S3 deductions
D6	Standard error envelope <code>{ "error": { "code", "message", "details" } }</code>	All endpoints (Section 4)
D7	<code>?page=</code> / <code>?per_page=</code> pagination on list endpoints	All <code>GET</code> list endpoints (Section 4)
D8	<code>redis-bus-subscriber</code> worker present on every system	Running but idle on S2 (Section 8.2)
D9	Service-Key authentication for inter-system calls	<code>ServiceKeyAuthenticate</code> middleware — used by inbound <code>POST /employees/{id}/deductions</code> (ID12) and outbound <code>POST /journal-entries</code>
D10	Structured JSON logging	Standard Laravel logging channel configuration (S0 §10)
D11	Health-check endpoint	<code>GET /api/v1/health</code> (S0 §10)
D12	Cross-system contract test pairs	S2 ↔ S1 (employee lifecycle events), S2 ↔ S4 (journal entries, payroll/severance/leave events), S3 ↔ S2 (deductions push) — Section 11
D13	Soft delete / Dead File retention	<code>employees.archived_at</code> , <code>offboarding_records</code> (Section 9.5)
D14	(platform-wide convention, see S0 §9)	Applies uniformly; no S2-specific deviation

10.2 Environment configuration (`.env`)

```
# Database
DB_DATABASE=wh_s2_db
DB_USERNAME=wh_s2_app
```

```
DB_PASSWORD=<secret>

# JWT / Gateway (S0 §4)
JWT_PUBLIC_KEY_PATH=/secrets/jwt/public.pem
SERVICE_KEY=<secret>

# Redis / event bus (D2, D8)
REDIS_HOST=redis.internal
REDIS_PORT=6379
EVENT_OUTBOX_PUBLISH_INTERVAL=10

# Payroll configuration (from M3)
WORKING_DAYS_PER_MONTH=26
WORKING_HOURS_PER_DAY=8
PENSION_EMPLOYEE_RATE=0.07
PENSION_EMPLOYER_RATE=0.11

# Leave & attendance configuration (from M4)
EXCLUDE_WEEKENDS_FROM_LEAVE=true
ANNUAL_LEAVE_BASE_DAYS=16
ANNUAL_LEAVE_MAX_DAYS=30

# Idempotency (D5)
IDEMPOTENCY_KEY_TTL_HOURS=48

# Object storage (employee photos, guarantor letters, payslips)
FILESYSYSTEM_DISK=s3
AWS_BUCKET=wh-s2-documents
```

CROSS-REFERENCE

For the full deployment topology — container layout, nginx gateway routing rules, secret management, and shared infrastructure (Redis, MySQL hosts) — see **S0 §10 (Deployment & Operational Conventions)**.

11. Non-Functional Requirements & Testing

Category	Requirement
Availability	S2 must be available during business hours (06:00–23:00 local) with 99.5% uptime; the <code>RunLeaveAccruals</code> job (00:30) and <code>PublishOutboxEvents</code> / <code>PurgeIdempotencyKeys</code> jobs may run during a low-traffic maintenance window
Latency	<code>GET /employees</code> , <code>/payroll-runs</code> , <code>/leave-requests</code> (list endpoints) must respond in < 300ms at p95; <code>POST /payroll-runs</code> (run generation for up to 300 employees) must complete in < 10 seconds
Throughput	Must support at least 50 concurrent payroll-officer/HR-manager sessions during month-end payroll processing
Security	All write endpoints involving money (payroll approval, loan disbursement, severance) must be Idempotency-Key protected (D5) and logged with the acting <code>employee_id</code> for audit; inbound S3 deduction calls must be Service-Key authenticated (D9)
Data integrity	<code>payroll_entries.net_pay</code> must always equal <code>gross_pay - total_deductions</code> ; <code>loan_records.remaining_balance</code> must never go negative; <code>leave_balances.days_remaining</code> must always equal <code>days_accrued - days_used</code>
Testing — unit	<code>TaxCalculatorService</code> , <code>OvertimeCalculatorService</code> , <code>PensionService</code> , <code>SeveranceService</code> , and <code>LeaveAccrualService</code> each require unit tests covering every bracket/category/schedule row in Sections 5.3–5.7
Testing — feature	Full Laravel feature tests for every endpoint in Section 4, covering permission checks (Section 6) and lifecycle transitions (Section 7)
Testing — contract	S2's <code>openapi.json</code> validated against the sync-call contracts in S0 §5.3: <code>POST /journal-entries</code> (outbound to S4), <code>POST /employees/{id}/deductions</code> (inbound from S3), <code>GET /employees / /payroll-runs / /leave-requests / /attendance-records</code> (inbound from S4)
Testing — event integration	Per D12, contract test pairs: S2 ↔ S1 (<code>employee.created</code> / <code>.updated</code> / <code>.archived</code>), S2 ↔ S4 (<code>payroll_run.approved</code> , <code>severance.calculated</code> , <code>leave.approved</code> , plus the two synchronous journal-entry posts), S3 ↔ S2 (<code>employees/{id}/deductions</code>)
Testing — UAT	Tracked in S4's Requirements Traceability Matrix (RTM) alongside the other three systems

12. Integration Contract Summary

12.1 What S2 provides

To	What	Reference
S1	Employee lifecycle events (<code>employee.created</code> / <code>.updated</code> / <code>.archived</code>) for user-account provisioning, display-name sync, and deactivation	Section 8.1, ID1, ID2
S4	Journal entries for payroll approval, severance, and loan disbursement (synchronous, idempotent); <code>payroll_run.approved</code> / <code>severance.calculated</code> / <code>leave.approved</code> events for cache invalidation; cached read endpoints for employees, payroll runs, leave requests, attendance	Section 5.1, 5.6, 8.1, 8.3
S3	<code>POST /employees/{id}/deductions</code> endpoint for staff meal charges	Section 4.2, ID12

12.2 What S2 depends on

From	What	Reference
S1	JWT verification for every inbound request (via the gateway); <code>permissions[]</code> claims for authorization	S0 §3–4, Section 6
S3	Timely delivery of staff meal deductions before payroll-run generation each month (operationally — not a hard dependency, since deductions arriving after a run is generated simply apply to the next run)	Section 4.2, ID12

12.3 Build-order implication

Because S1 issues every JWT that S2's endpoints require, **a stub or real S1 deployment must exist before S2's endpoints can be exercised end-to-end** — the same dependency noted in S1 §12. Within S2 itself, the HR module should be built first (it has no internal dependencies), followed by Leave & Attendance (depends on `employees`), followed by Payroll (depends on both `employees` and `leave_balances` / `overtime_records` per ID6). The S2 ↔ S1 employee-lifecycle event contract (Section 8.1) and the S2 ↔ S4 journal-entry contract (Section 8.3) should be agreed with the S1 and S4 developers early, since both are exercised from S2's very first HR and payroll endpoints.

End of Document — S2, Workforce & Payroll, v1.0